

Asymptotic

Incremental repository audit

Remainder-certificate provenance, affine-recognition complexity,
and consistency of acceptance evidence

Reviewed repository VladimirReshetnikov/Asymptotic

Pinned revision 651f2029d0b2cd4da9e4dfdf1f4275a124d23b99

Review date September 10, 2026

Abstract

This report examines the current implementation against the repository’s existing four-wave review corpus, rather than presenting its established backlog as new work. Three additional finding groups survive that exclusion process. First, a second, no-op truncation retains a transported quantitative remainder contract but loses the companion expression that the contract explicitly names. A focused public Wolfram-kernel witness confirms this inconsistency; it does not show an incorrect error bound. Second, the newly added exact-affine fast path repeatedly reclassifies overlapping nonlinear subtrees. For a Horner expression with $4n + 1$ tree nodes, the inspected control flow performs exactly $4n(n + 1)$ affine recognizer visits. A separate exact-rational model reproduces the count and preserves interval endpoints under request-local classification reuse. Third, the new Mathics acceptance checker accepts synthetic receipts with contradictory entry-point or source-stability metadata, despite checking their source maps and protocol records successfully.

The accompanying artifacts include a source-matched candidate truncation patch, a candidate receipt guard, an independent interval prototype, runnable Python checks, and Wolfram regression specifications. The executed Python run has 34 passing test cases. Native observations, synthetic records, source proofs, and unrun acceptance specifications are kept separate throughout. This is a focused incremental audit, not a claim of exhaustive defect discovery or full-suite acceptance.

The snapshot is the unit of analysis. Later changes to `main` are outside the claims in this report. No remote repository changes were made.

Contents

1	Executive assessment	2
2	Snapshot, scope, and method	2
2.1	What was reviewed	2
2.2	Evidence categories	3
2.3	How novelty was decided	3
3	Architecture and the assessed boundaries	4
4	Novelty crosswalk	5
5	N01: a retained bound contract loses its discard companion	6
5.1	The public witness	6
5.2	Why the upper bound remains correct	6
5.3	The exact source mechanism	7
5.4	A minimal repair and its invariant	7
6	N02: repeated affine recognition is quadratic on Horner trees	8
6.1	The new fast path is mathematically useful	8
6.2	Where repeated work enters	8
6.3	An exact family and visit count	8
6.4	A semantics-preserving design	9
6.5	Important limits of the result	10
7	N03: acceptance receipts can contradict their own source claims	11
7.1	What the checker already verifies	11
7.2	Selected source versus fingerprinted source	11
7.3	Recorded first drift versus stability flags	12
7.4	Candidate guard and its limits	12
7.5	Why the finding is new rather than a runner replay	13
8	Implementation priorities and focused development	13
8.1	First: close the concrete no-op provenance gap	13
8.2	Second: make the new evidence consumer fail on contradictions	13
8.3	Third: remove repeated recognition without weakening certificates	13
8.4	What this plan intentionally does not repeat	14
9	Validation, artifacts, and reproducibility	14
9.1	Executed independent checks	14
9.2	Native reproduction and patch workflow	14
9.3	Artifact map	15
9.4	Limits that remain material	16
10	Conclusion	16
A	Source locations and review anchors	16
B	Acceptance assertions in compact form	17

1 Executive assessment

The most useful new work in this snapshot is not another general request for better caching, richer transseries, native compatibility, or stronger certificates. Those directions already appear in the existing reviews. The additional defects identified here occur at recently changed boundaries: a certificate copied after truncation, a speculative recognizer inserted before interval evaluation, and a postprocessor that consolidates test receipts. Each boundary can appear locally reasonable while violating a compositional property one level above it.

ID	Classification	New observation	Evidence obtained
N01	Medium, metadata integrity	A no-op truncation retains a TransportedThroughTruncation contract but drops TruncationDiscardedPart .	Pinned source and one successful public native witness.
N02	Medium, performance	Repeated failed affine recognition costs $\Theta(n^2)$ structural visits on an $O(n)$ Horner tree.	Source recurrence, exact model, endpoint-preserving prototype checks.
N03	Medium, validation integrity	The new receipt checker accepts selected-entry and recorded-drift contradictions.	Executed upstream-function excerpt against explicitly synthetic records; candidate-guard controls.

Table 1: Finding groups retained after comparison with the existing review baseline. Severity describes the demonstrated consequence, not a hypothetical worst case.

None of these observations establishes a newly incorrect asymptotic coefficient, a false rational interval enclosure, or an actually corrupted historical CI receipt. That distinction matters. N01’s stored upper bound remains valid; N02 is an algorithmic cost defect in a helper path; N03 is a demonstrated acceptance blind spot on artificial records. Inflating these conclusions would obscure both their real significance and the work still required to close them.

The immediate actions are correspondingly small and testable. Preserve the existing discard companion on the no-op branch. Compute affine classifications once per interval-evaluation request without weakening the exact-affine translation fix. Make the receipt checker reject a selected source that is not the intended fingerprinted entry, and reject a non-null record of observed source drift. Broader changes should follow only after these exact invariants have focused acceptance.

2 Snapshot, scope, and method

2.1 What was reviewed

The reviewed revision is 651f2029d0b2cd4da9e4dfdf1f4275a124d23b99. Its commit records the article’s updated account of affine ranges and truncation-bound transport. The maintained package is named **AsymptoticAnalysis**; the public inverse constructor remains **AsymptoticInverse**. The repository contains modular kernel sources and a generated standalone entry. The latter was successfully imported as a complete 685,212-character text and loaded through a string stream during this review. This observation concerns the pinned standalone file, not every possible loading mode. Source and development references are commit-pinned in the bibliography. [1, 2, 9]

The source examination covered the ordinary power-log representation and precision algebra, forward and inverse dispatch, explicit series operations, Lambert and reciprocal-log handling, Gamma/Barnes forward adapters, convergent Dirichlet and Lerch expansions, rational interval certificates, several Mathics compatibility modules, native-result boundaries described by the implementation register, and the new acceptance checker. Inspection was particularly detailed around the paths relevant to the retained findings. The audit does not claim that every line of every kernel module, test, vendor document, and external report was independently reread.

The exclusion baseline contains 36 review packages in four waves. The maintained register reports 231 attributed entries before overlap consolidation. That number is neither a count of distinct defects nor a count of outstanding problems. The register and the two later-wave intakes were used as the principal exclusion map, with targeted checks of the original nearby reports. In particular, report 14’s original affine-rounding and quantitative-bound findings were compared with their current repairs. Report 36’s ledger was checked to distinguish its runner and aggregation concerns from the new downstream checker examined here. [8, 9, 10, 11, 12, 13]

2.2 Evidence categories

Four categories are deliberately not combined into a single passing total. *Source-proved behavior* is a consequence of the inspected code, with the relevant admission and control-flow assumptions stated. *Native observation* means that a successful official-kernel tool call returned the reported public result. *Independent-model evidence* executes an explicitly separate model of an algorithm or mathematical identity. *Synthetic-validator evidence* executes validation functions against constructed records; it says nothing about whether Mathics actually ran those cases.

The bundled Wolfram files contain further acceptance specifications and helper instrumentation. Those files were not successfully executed as a suite in this review. Subsequent attempts to use the native service returned network/502 failures, including a simple version query. Such attempts contribute no test results. Earlier partial direct-URL loading attempts likewise do not count as successful package loads. The successful string-stream load and public N01 witness are recorded separately in `evidence/native-witness.json`.

2.3 How novelty was decided

A finding was excluded when its mechanism and proposed obligation were already represented in the prior corpus, even if it remained open. A changed witness alone was not treated as novelty. Conversely, a defect introduced by a repair can be new when it concerns a newly added field, call path, or verifier obligation that did not exist in the reviewed baseline. The retained findings meet this narrower criterion; their nearest prior items remain explicitly attributed.

This method avoids two common distortions. A fixed historical issue is not reclassified as a current defect merely because its original report is still in the repository. A general old recommendation is not renamed and presented as a new architectural proposal. The development plan in Section 8 therefore concerns only the specific new obligations demonstrated below.

3 Architecture and the assessed boundaries

The ordinary representation separates a positive local coordinate, a finite jet, and a remainder class. Explicit calculus uses an expression of the form

$$b(x) + p(x) \left(\sum_j w(x)^{\beta_j} C_j(\log w(x)) + R(w(x)) \right), \quad w(x) \rightarrow 0^+,$$

where b is an offset and p is an exact carrier. The jet records retained rows, a power frontier, and a logarithmic degree. This is substantially more structured than a bare finite expression. It explains why preservation of metadata is part of correctness of the object API, even when a displayed approximation is unchanged.[2, 3]

The inspected arithmetic already accounts for precision loss in multiplication, complete coefficient rows, and logarithmic degrees. The current register also records focused repairs for real-coefficient admission, retained assumptions, semantic equality of exponents, native-tail import/export, and separate Fourier termination. These repairs are the baseline of this report, not additional findings. Their recorded acceptance counts remain scoped to their own snapshots and selected suites.[9, 10]

Boundary	Role in the implementation	Outcome of this audit
Ordinary jet algebra	Retained power-log blocks and inherited precision are combined before public result construction.	No additional coefficient counterexample established in the inspected paths.
Explicit calculus	Transports offsets, carriers, jets, assumptions, domains, and recipes.	N01 concerns the new quantitative-bound companion-copy branch.
Lambert and reciprocal-log paths	Keep exact cores or carriers and use a separate small inverse-logarithmic coordinate.	Existing closure, domain, and cutoff concerns were excluded rather than rediscovered.
Gamma/Barnes forward normalization	Positive special-function factors are combined in the real logarithmic domain, with bounded simplification.	No new identity error established in the inspected normalization paths.
Dirichlet/Lerch providers	Supply explicit coefficient and quantitative-tail metadata from convergent defining sums and moment bounds.	A Zeta result gives a transparent N01 witness; the provider's bound is not the defect.
Exact certificate engine	Uses rational interval endpoints and outward rounding; the new affine path delays rounding until an exact range is found.	N02 identifies repeated recognition work, not unsound rounding.
Mathics adapters	Supply bounded package-local compatibility operations and avoid modifying the global System implementations.	Previously reported semantic and resource limitations remain excluded.

Boundary	Role in the implementation	Outcome of this audit
Acceptance postprocessor	Matches recorded inputs, protocol output, cases, versions, and coverage against a reference.	N03 finds missing consistency checks between fields that the postprocessor otherwise validates separately.

The separate mathematical scales are intentional, not themselves evidence of a missing universal representation. Likewise, a native formal expansion is not silently treated here as an analytic remainder theorem. Those distinctions are already explicit in the maintained register and related notes. This review does not count the known native-superset, Mathics-completeness, or vendored-corpus goals as newly discovered unimplemented features.[9, 10, 11]

4 Novelty crosswalk

New ID	Nearest prior item	What is different here
N01	R14 N05; maintained C22	The original quantitative bound is now preserved. The newly introduced TruncationDiscardedPart field is subsequently omitted by a no-op metadata copy, leaving its retained contract incomplete.
N02	R14 N02; maintained C20; general P08	Exact affine evaluation before rounding is now implemented. Its recursive placement causes a new, explicitly derived quadratic recognizer count on nested nonlinear expressions.
N03	W4-09, W4-12, W4-14; R36 N06	The downstream checker is a later implementation. The counterexamples concern contradictions in its saved-record validation, not rediscovery of source-freezing, package-load, or process-aggregation gaps.

Table 3: Attribution without repeating the old recommendation as a new finding.

The relevant distinction for N01 is especially narrow. Report 14 already showed that quantitative tail metadata could be lost by truncation and proposed transport through discarded known terms. The current code implements that transport. The present report neither repeats the old missing-bound witness nor asks again for that general feature. It demonstrates that the repair’s new companion field is not included in the repair’s own no-op copy list. [12, 3, 9]

N02 also preserves the earlier repair’s purpose. Report 14’s affine example shows why rounding a huge translated constant before cancellation can destroy a useful certificate. The new performance proposal must retain exact affine cancellation; removing the fast path would regress the old finding. The new analysis concerns how often the exact classifier is invoked after that fast path was inserted. [12, 5]

For N03, repository history identifies the addition of `validation/check_mathics_acceptance.py` at commit `e10ef1b71f5b55080c62f52c3ce9d3d6e039fc99`, whose message describes verifying and recording complete focused Mathics acceptance. The inspected validator therefore has a concrete implementation and a separate proof obligation. The synthetic counterexamples do not allege a defect in the specific acceptance record added by that commit.[6, 7]

5 N01: a retained bound contract loses its discard companion

5.1 The public witness

Use the package backend explicitly, so native fallback is not an alternative explanation of the result:

```
s = AsymptoticAnalysis[AsymptoticExpansion[
  Zeta[x], {x, Infinity, Log[4]}, "Backend" -> "Package"];
t = AsymptoticAnalysis[SeriesTruncate[s, Log[2]]];
u = AsymptoticAnalysis[SeriesTruncate[t, Log[2]]];

{t["TruncationDiscardedPart"],
 u["TruncationDiscardedPart"],
 t["AbsoluteRemainderBound"] == u["AbsoluteRemainderBound"],
 u["ForwardRemainderContract"]["Type"], Normal[t], Normal[u]}
```

The successful native tool call returned three `GeneralizedSeries` objects. After alpha-renaming its generated variable to x , the relevant observation is

```
{2^(-x) + 3^(-x),
 Missing["KeyAbsent", "TruncationDiscardedPart"],
 True,
 "TransportedThroughTruncation",
 1, 1}
```

This call also emitted `Power::infy` and `Greater::nord` diagnostics. Their origin was not isolated, and they are not counted as a separate finding. The stored-field comparison, object heads, and unchanged expressions are the specific observations used here. The service had separately identified itself as Wolfram 15.0.0 on Linux; that version was not re-emitted inside this witness's output. The evidence file preserves these qualifications.

5.2 Why the upper bound remains correct

For real $x > 1$, the provider used by this example stores the first three Dirichlet terms and a positive omitted tail:

$$\zeta(x) = 1 + 2^{-x} + 3^{-x} + R_4(x), \quad 0 < R_4(x) \leq B_4(x) := 4^{-x} \left(1 + \frac{4}{x-1} \right).$$

The bound follows from the first omitted term plus the integral of t^{-x} from 4 to infinity. It is also the formula implemented by the inspected provider. [4]

The first truncation discards

$$\Delta(x) = 2^{-x} + 3^{-x}$$

and retains the approximation 1. The current transport code correctly forms

$$B_t(x) = B_4(x) + |\Delta(x)|.$$

Consequently $|\zeta(x) - 1| \leq B_t(x)$ under the retained conditions. The second truncation discards no additional row and keeps the same B_t . Nothing in this witness makes B_t false or changes the power-log remainder. The defect is that the second result still advertises a contract whose statement refers to `TruncationDiscardedPart`, although that property is now missing.

5.3 The exact source mechanism

In `src/Kernel/SeriesOperations.wl`, `seriesTransportRemainderBound` implements the transport. The nontrivial branch computes the removed rows, constructs their expression, and attaches both a bound and a companion. The following excerpt omits unrelated fields and abbreviates the statement:

```
"AbsoluteRemainderBound" ->
  data["AbsoluteRemainderBound"] + Abs[discarded],
"TruncationDiscardedPart" -> discarded,
"ForwardRemainderContract" -> <|
  "Type" -> "TransportedThroughTruncation",
  "OriginalAbsoluteRemainderBound" -> data["AbsoluteRemainderBound"],
  "OriginalContract" -> Lookup[data, "ForwardRemainderContract", ...],
  "Statement" -> "... plus Abs[TruncationDiscardedPart] ..."
|>
```

When no rows are removed, however, the helper builds a fresh result and copies only a fixed list of optional keys. That list contains `ForwardRemainderContract` and the quantitative bound, but not `TruncationDiscardedPart`. The previous companion is neither regenerated nor copied. The public truncation wrapper invokes this helper after constructing the new result, so the omission is directly reachable.^[3]

The prerequisite is an already transported object. Applying a no-op to an original Zeta expansion need not have a discard companion at all. A regression must therefore exercise the two-stage sequence, rather than merely checking whether a single truncation preserves `AbsoluteRemainderBound`.

5.4 A minimal repair and its invariant

Add `TruncationDiscardedPart` to the existing no-op copy list. The guard that copies only keys actually present should remain unchanged. This preserves the companion when it exists and does not invent a history field for an original untransported result. The nontrivial branch should continue computing its newly discarded part rather than reusing a previous one.

The desired invariant is a scoped form of idempotence. Let T_h be truncation at cutoff h , and suppose a second application removes no more rows. Then the finite expression and remainder should remain unchanged, and any copied transported-bound contract should retain the companion on which its statement depends. This does not require byte-for-byte equality of whole objects: a new recipe wrapper or other provenance may legitimately differ.

The supplied patcher performs this one-list change only when the pinned-source text matches exactly once. It defaults to showing a diff and writes only with `-write`. It changes the modular source; the generated standalone must be rebuilt by the repository's own builder. Python fixtures check matching, line-ending preservation, and refusal of unmatched or repeated patches. These fixtures validate the patcher's operation, not the patched Wolfram package.

The six supplied MUnit specifications check companion preservation, unchanged bounds, unchanged expression/remainder, repeated no-ops, absence of fabricated history on an original object, and retained contract type. They have not been executed as a native suite in this review. Focused native acceptance of the patch is therefore still required.

6 N02: repeated affine recognition is quadratic on Horner trees

6.1 The new fast path is mathematically useful

The certificate engine represents an enclosure by two exact rational endpoints and rounds outward to a dyadic grid. Its new affine recognizer returns a pair (a, b) for an expression equal to $ax + b$, provided that the expression is built from rational constants, the source symbol, addition, and multiplication of admitted affine factors. A product is refused when two accumulated factors are both nonconstant. An admitted affine expression is evaluated on an interval I by the exact range

$$\text{range}(ax + b, I) = [\min_{z \in \partial I}(az + b), \max_{z \in \partial I}(az + b)],$$

followed by one outward rounding step.[5]

This fixes a genuine conditioning problem in expression evaluation: at a huge rational translation, rounding x and the translation independently can lose cancellation that is exact in the original affine function. The performance problem discussed here is not an objection to that mathematics.

6.2 Where repeated work enters

`certEnclose` first tries `certAffineRange` on every nontrivial `Plus` or `Times` node. If the recognizer fails, ordinary interval evaluation recurses into the children. Those children then try the same affine recognizer on their own subtrees. Meanwhile, `certAffinePair` itself maps recursively over all the children before combining their classifications. The same nested subtree can therefore be classified repeatedly, once for each nonlinear ancestor.[5]

A claim that one call of `certAffinePair` is linear in its input tree does not imply that the surrounding evaluator is linear. The composition of an $O(m)$ speculative recognizer with a recursively descending fallback needs its own cost analysis.

6.3 An exact family and visit count

Define

$$e_0(x) = x, \quad e_n(x) = x(1 + e_{n-1}(x)), \quad n \geq 1.$$

This is the ordinary nested Horner form of $x + x^2 + \dots + x^{n+1}$; no expansion into monomials is needed to construct it. Count one node for each operator, constant occurrence, and variable occurrence in the expression tree. Then

$$N_n = 4n + 1.$$

For $n \geq 1$, e_n is nonlinear. The intermediate sum $1 + e_0$ is affine, while $1 + e_{n-1}$ is nonlinear for $n \geq 2$.

Let A_n be the number of `certAffinePair` visits made by one recognition attempt on e_n . Since the current recognizer maps all child expressions,

$$A_0 = 1, \quad A_n = A_{n-1} + 4 = 4n + 1.$$

Similarly, recognition of $1 + e_{n-1}$ costs

$$B_n = A_{n-1} + 2 = 4n - 1.$$

Proposition 1. *For the inspected control flow, evaluating an enclosure of e_n performs exactly*

$$C_n = 4n(n + 1)$$

affine-recognizer visits, provided that the source expression retains the stated nested grammar and the evaluation reaches these helpers.

Proof. At e_n , the evaluator first makes the unsuccessful recognition attempt costing A_n . It then evaluates the outer variable directly and visits $1 + e_{n-1}$. Recognition of that sum costs B_n . For $n \geq 2$ it also fails, so the evaluator continues into e_{n-1} . For $n = 1$, the sum is affine and the recursion stops. Thus $C_1 = A_1 + B_1 = 8$, and

$$C_n = C_{n-1} + A_n + B_n = C_{n-1} + 8n.$$

Summation gives $C_n = 8 \sum_{j=1}^n j = 4n(n + 1)$. Setting $C_0 = 0$ makes the same recurrence valid at the first step as well. $\square$

In terms of tree size,

$$C_n = \frac{N_n^2 + 2N_n - 3}{4}.$$

This is a quadratic number of structural recognition visits. It is not a quadratic bit-complexity theorem for all rational arithmetic, nor a wall-clock benchmark of **InverseCertificate**.

Horner depth n	Tree nodes $4n + 1$	Old recognizer visits	Prototype classifications
1	5	8	4
2	9	24	7
4	17	80	13
8	33	288	25
16	65	1,088	49
32	129	4,224	97
64	257	16,640	193

Table 4: Executed independent-model counts. The prototype shares the variable leaf by node identity, hence $3n + 1$ unique classifications; without that sharing, one classification per tree node still gives a linear structural bound.

6.4 A semantics-preserving design

A request-local bottom-up classification pass can compute each node’s affine pair, or a failed classification, once. The enclosure pass then consults those results. An affine node still uses its exact range and one outward rounding. A nonaffine node still uses the same recursive interval operations in the same order. This is a targeted elimination of repeated recognition, not a global symbolic-expression cache.

```
def classify(node):
    if node.identity in local_classifications:
        return local_classifications[node.identity]
    child_pairs = [classify(child) for child in node.children]
    pair = combine_affine_pairs(node, child_pairs)
    local_classifications[node.identity] = pair # pair or failure
    return pair
```

```
def enclose(node, interval, rounding_context):
    pair = local_classifications[node.identity]
    if pair is not failure:
        return round_outward(exact_affine_range(pair, interval))
    return original_interval_fallback(node, interval, rounding_context)
```

The implementation must not recursively hash or serialize the entire subtree on every cache lookup, since doing so can reintroduce the same structural cost. The supplied Python model uses live node identity and retains the map only for one request. A Wolfram implementation could use a bounded annotated traversal or stable local node identifiers; it needs its own measurement of actual allocation and lookup behavior rather than assuming that association keys are free.

The interval-equivalence argument is a structural induction. For a rational constant or the source symbol, both evaluators return the same enclosure. If a composite node is affine, both classifiers compute the same exact pair and both evaluators round the same exact range. If it is not affine, the induction hypothesis gives equal child enclosures, and the same ordered arithmetic folds give equal parent enclosures. No stronger interval theorem or new rounding rule is introduced by retaining classifications.

The supplied model includes dyadic outward rounding, rather than comparing only unrounded symbolic values. It checks identical intervals on seven Horner depths, 120 seeded random trees, and the huge exact-affine translation control at three arithmetic precisions. Separate tests verify that the rounding helper encloses 2,982 rational values across the selected bit counts. These tests support the prototype's stated grammar; they do not validate the entire Wolfram certificate engine.

6.5 Important limits of the result

The exact count is conditional on the nested tree reaching `certEnclose` without prior flattening or polynomial expansion into another shape. Some public certificate preparation paths simplify or expand expressions, and different paths may evaluate a derivative or residual rather than the user's original tree. A successful end-to-end native performance witness was not obtained here. The finding is therefore a source-proved helper complexity defect with a reproducible family, not an asserted timing regression for every public certificate call.

Likewise, one classification per node is an $O(N)$ structural-operation bound, not necessarily $O(N)$ bytes or bit operations. Exact rational pairs can grow in size, and storing pairs at several levels can retain more coefficient data than a single traversal would. A production implementation should measure that cost and bound any optional retained state. None of these qualifications changes the quadratic repeated-visit count of the current control flow.

A focused native acceptance plan should instrument recognizer calls, compare returned rational intervals exactly, retain affine translations and ordinary nonlinear controls, and then benchmark a public path shown to preserve the adversarial tree. Timing only a separately reimplemented classifier would not establish a speedup of `InverseCertificate`.

7 N03: acceptance receipts can contradict their own source claims

7.1 What the checker already verifies

The new checker is considerably more than a search for a passing count. Its `validate_shard` function requires a Mathics runtime, completion, source-stability and fresh-kernel flags, consistent selected/executed/succeeded counts, zero failed/not-run counts, the expected source set and fingerprints, a matching suite snapshot, and matching configured and effective iteration budgets. Optional final fingerprints must agree with the initial map. Each case must have the expected identity and group, a successful outcome and zero exit code; raw protocol fields must agree with stored fields and the expected Mathics version. The parsed printed actual and expected values must also agree. The outer checker combines shard coverage against a Git-derived reference.[6]

Those are useful controls and should remain. The new finding is that some individually plausible fields are never related to one another strongly enough. The independently executed excerpt in this bundle contains the upstream `require`, `canonical_sources`, `protocol_fields`, and `validate_shard` functions. It intentionally excludes the Git-discovery and CLI layers. An additional supplied script can compare the four function ASTs against a complete local checkout; that optional source-matching run was not performed here. The tests therefore establish what the copied shard validator accepts, not that an artificial bundle was run through every outer workflow command.

7.2 Selected source versus fingerprinted source

The checker first requires the selected basename to be the actual package entry, then infers the layout from the immediate parent directory:

```
parts = source.replace("\\", "/").split("/")
require(parts[-1] == ENTRY, "Unexpected package entry filename")
layout = "modular" if len(parts) >= 2 and parts[-2] == "Kernel" else "standalone"
```

It then independently canonicalizes keys in `TestedSourcesSHA256` and checks the resulting relative-name/digest dictionary against the expected dictionary. For kernel files, canonicalization retains the final filename and the fact that its parent is named `Kernel`; it discards the earlier path components. There is no subsequent requirement that the selected source path identify the fingerprinted entry path.[6]

Start with an otherwise valid synthetic modular shard. Change only

```
report["Source"] = "/NOT-FINGERPRINTED/src/Kernel/AsymptoticAnalysis.wl"
```

while leaving the fingerprint map under `/checkout/`. The copied upstream validator accepts the shard. The same mutation is accepted for a standalone source when the basename remains `AsymptoticAnalysis.wl`. In neither case does the saved map contain the selected path.

The existing filename check is important: selecting `/checkout/src/Kernel/InverseCertificates.wl` is *already rejected*. The bundled test retains that case as a negative control, not as a finding. N03 concerns two differently located files with the *same admitted entry basename*, not arbitrary helper-module admission. Matching an entry name and matching an independently supplied digest map do not establish that the two records describe the same selected path.

7.3 Recorded first drift versus stability flags

The current review baseline records a runner repair that latches the first observed mismatch even when later source content is restored. The point of that record is monotonic invalidation: a later equal endpoint must not erase the fact that a mismatch was already seen.^[11]

The new checker requires a true source-stability flag and permits final fingerprints to be absent or equal to the initial map. It does not inspect `FirstObservedSourceDriftSHA256`. Accordingly, an artificial receipt with true stability flags, equal initial and final maps, and a non-null first-drift map is accepted by the copied upstream validator. An empty dictionary in that non-null field is also accepted; the candidate guard rejects both because the current runner’s no-drift sentinel is `None`.

This is not another demonstration that endpoint fingerprints fail to prove continuous immutability. It is narrower: when a record already contains explicit evidence of observed drift, the validator should not accept mutually inconsistent stability claims. Nor is it an allegation that the repository’s real runner ever emitted the particular contradictory fixtures. The tests deliberately construct them to exercise the checker’s consistency contract.

Synthetic record	Upstream excerpt	Candidate guard
Consistent modular positive control	Accepts	Accepts
Consistent standalone positive control	Accepts	Accepts
Selected entry absent from the fingerprint map	Accepts	Rejects
Fingerprint-listed helper selected as package entry (existing control)	Rejects	Rejects
First observed drift recorded despite equal endpoint maps	Accepts	Rejects
Existing failure controls: bad hash, incomplete flags, duplicate/empty cases, missing protocol marker	Rejects	Rejects

Table 5: Executed synthetic shard-validation behavior. No row is a real Mathics execution receipt. The synthetic version strings explicitly identify the host as artificial, and the fingerprints are dummy fixture data.

7.4 Candidate guard and its limits

The supplied preflight guard adds two cross-field checks before delegating to the original shard validator. The selected source’s lexically normalized recorded path must identify exactly one key in the fingerprint map. A present `FirstObservedSourceDriftSHA256` must be null. The existing entry-basename check is preserved; it is not claimed as a new repair.

Path normalization is deliberately lexical. Windows drive and UNC syntax use Windows case normalization; POSIX paths remain case-sensitive. The guard does not resolve a receipt’s paths on the reviewer’s filesystem. Such resolution could refer to unrelated files and would not establish the identity of a file on the recorded execution host anyway. The current bundle does not claim symlink, hard-link, filesystem-lock, or executed-byte attestation from this comparison.

A production integration should first check that the repository’s real receipts use a consistent selected-path spelling, then add fixtures for both layouts, relative and absolute path spellings that are intentionally supported, and legacy receipts with an absent drift field. If a supported entry format is extended later, its admission rule and dependency contract should be updated together. A supported new entry must remain linked to its own fingerprinted path.

These guards cannot make a maliciously fabricated receipt authentic. A party able to rewrite every field can fabricate consistent fields as well. The value is detection of accidental or adversarial *internal contradictions* under the checker's existing trust model. No signature, trusted-execution mechanism, or cryptographic claim is proposed here.

7.5 Why the finding is new rather than a runner replay

The earlier wave-4 findings include live-source capture, source closure, first-drift retention, mandatory package loading, and coverage aggregation. Their recommended runner repairs should not be relabeled as new work. The present counterexamples exercise a later checker that reads already saved records and can accept contradictions those records make visible. They can be reproduced without launching a child process, modifying a source file during execution, loading Mathics, or bypassing a failed test. The additional obligation is at the consumer of evidence, not its producer.[\[11, 13, 7\]](#)

8 Implementation priorities and focused development

8.1 First: close the concrete no-op provenance gap

Apply the source-matched N01 change in the modular file, rebuild the standalone, and run the six supplied native specifications together with the repository's existing truncation-bound tests. Preserve the original quantitative bound and conditions exactly on a no-op. Retain original-source objects without adding a spurious discard history. The acceptance claim should name those selected tests and the rebuilt artifact rather than imply complete package acceptance.

A useful immediate development check is contract dependency closure for this one contract type: whenever a result stores `Type -> "TransportedThroughTruncation"`, access to the companion expression used by its current statement should succeed. This is a narrow executable invariant for a new metadata pair, not a proposal to redesign the entire result schema or serialization system.

8.2 Second: make the new evidence consumer fail on contradictions

Integrate the N03 checks beside the existing shard checks, using the real receipt layout as a compatibility control. Keep the current checks for fingerprints, protocol consistency, execution budgets, and case coverage. A negative fixture should state which single invariant it violates. This isolates regressions and prevents a fixture rejected for an unrelated malformed field from falsely appearing to test the desired guard.

The minimal test matrix has two valid entry layouts and two contradiction families: an unfingerprinted selected path and a recorded first mismatch contradicting the stability flag. A fingerprinted nonentry module remains an existing rejection control. The supplied tests also retain the old bad-hash, duplicate-case, empty-case, and protocol controls. The outer CLI's complete-coverage check remains separate; the new guards should not change how expected case coverage is computed.

8.3 Third: remove repeated recognition without weakening certificates

Implement a request-local classification pass or equivalent bounded annotated traversal for N02. Before timing, count recognizer visits and compare all returned rational endpoints exactly on the supported

grammar. Preserve the huge affine translation witness and valid nonlinear controls. Only then measure a public certificate path whose expression shape is known to reach the changed helper.

The later development opportunity is to reuse a node's exact affine classification across multiple interval evaluations within one certificate request, because the pair depends on the rational expression and source symbol, not on the interval. This extension needs an explicit request lifetime and must not reuse a classification across a changed source expression, variable, or unsupported grammar. It is not required for the initial repair, and no native speedup for it is claimed. The prototype already shows the smaller first step: one classification per node in a single enclosure call.

8.4 What this plan intentionally does not repeat

The existing register remains the place for the native-superset objective, Mathics semantic compatibility, general refinement demand propagation, multirate transseries, broad provenance compaction, and general proof-record architecture. Those are substantial goals but not new findings here. The three focused changes above can be reviewed and accepted independently without reopening every older design decision.[\[9, 10, 11\]](#)

9 Validation, artifacts, and reproducibility

9.1 Executed independent checks

The saved Python 3.13.5 run reports **34 passed** pytest cases. Fourteen cases cover the exact interval model: seven depths, three affine-translation controls, three parameterized rounding checks, and one seeded random-tree test. Fifteen cases exercise the copied validator and candidate guards. Five cases exercise the source-matched patcher's text handling and refusals. Some tests contain multiple mathematical checks, but these are not inflated into an additional package-test count.

Run from the extracted bundle root:

```
python -m pytest -q tests
python code/generate_evidence.py
```

The second command writes `evidence/independent-results.json`. The saved text version contains the same independent observations in a convenient readable form. The reported affine quantities are operation counts, not native timing measurements. The test duration printed by pytest is the duration of the Python checks, not a performance result for the Wolfram package.

9.2 Native reproduction and patch workflow

Use a local checkout of the pinned revision and a fresh official kernel. The supplied script accepts either the modular or the generated standalone entry:

```
wolframscript -file code/probe_native.wls /absolute/path/AsymptoticAnalysis.wl
```

It records the selected path and expected review revision, but explicitly does not attest the source bytes or Git state. Besides the N01 witness, it contains optional helper instrumentation for N02. That

instrumentation was not successfully executed during this review. Its counters should be inspected alongside the actual returned intervals before being used as native evidence.

To inspect and apply the N01 candidate locally:

```
python code/apply_truncation_fix.py /path/to/checkout
python code/apply_truncation_fix.py /path/to/checkout --write
# From the checkout root, rebuild its standalone artifact:
python validation/build_standalone.py
```

After loading the chosen rebuilt entry, run `TestReport` on `code/NativeRegressions.wlt`. The baseline should fail the companion-preservation assertions. A passing patched run is still an acceptance task for the maintainer; this bundle does not contain such a native receipt.

9.3 Artifact map

Path	Purpose and evidence status
article/review.tex, article/review.pdf	Self-contained LaTeX source and compiled report.
code/apply_truncation_fix.py	Opt-in, source-matched candidate N01 patch; patcher fixtures executed, native package acceptance unrun.
code/NativeRegressions.wlt	Six post-fix native regression specifications; unrun as a suite.
code/probe_native.wls	Reproduction script and native helper instrumentation; script not accepted as an executed test run here.
code/affine_model.py	Independent bounded-grammar exact-rational and dyadic interval model, including classification reuse.
code/receipt_excerpt.py	Copied upstream validation functions with source attribution; Git and CLI layers intentionally omitted.
code/receipt_guard.py, code/synthetic_receipts.py	Candidate consistency guard and explicitly artificial input records.
code/check_upstream_excerpt.py	Optional AST comparison against a supplied local checkout; not run against a complete checkout here.
code/generate_evidence.py	Recomputes operation counts and synthetic validation observations.
tests/	Executed Python tests and import configuration.
evidence/native-witness.json	Manual connector transcription, with variable normalization, diagnostic and version-capture limitations.
evidence/independent-results.json	Executed independent-model and synthetic-validator observations; not a package receipt.

Path	Purpose and evidence status
evidence/pytest-results.txt	Saved 34-case Python test output.
evidence/novelty-ledger.json	Nearest prior findings, new mechanism, and explicit nonclaims for each retained finding.

9.4 Limits that remain material

No full repository suite was executed. No fresh Mathics package run was obtained. The official-kernel evidence is a successful complete-load smoke and one public truncation witness, not a combined MUnit acceptance run. The new interval prototype covers a rational algebraic grammar, not transcendental enclosure functions or all certificate domains. The receipt tests execute the specified upstream shard-validation functions, not the full Git/CI acquisition workflow. The novelty check uses the maintained consolidated inventory and targeted original reports; it is not an independent line-by-line re-review of every old article. These boundaries are reflected in the machine-readable evidence as well as the prose.

10 Conclusion

The current snapshot contains three additional, sharply bounded opportunities for improvement. Preserve the companion of a transported bound when truncation is a no-op. Avoid repeatedly proving that the same nonlinear subtrees are not affine. Reject source-entry and source-stability contradictions when consolidating acceptance evidence. Each change preserves an existing useful mechanism rather than replacing it: quantitative bound transport, exact affine cancellation, and strict receipt checking.

The broader lesson is compositional. A correct bound formula is not a complete metadata-copy policy. A linear recognizer is not necessarily a linear enclosing algorithm. A set of individually checked receipt fields is not necessarily a consistent receipt. The supplied witnesses, structural proof, independent models, and targeted specifications make those three distinctions actionable without recasting the repository's older backlog as new work.

A Source locations and review anchors

The function names below refer to the pinned repository files and provide searchable locations without relying on line numbers of escaped connector responses. A moving branch should not be used to reconcile a mismatch.

Finding	Pinned source	Relevant location
N01	src/Kernel/SeriesOperations.wl	Bound-transport helper and following public truncation wrapper.
N01	src/Kernel/DirichletSpecialFunctions.wl	Zeta construction, bound, source coordinate, and metadata.

Finding	Pinned source	Relevant location
N02	<code>src/Kernel/InverseCertificates.wl</code>	<code>certAffinePair</code> , <code>certAffineRange</code> , and <code>certEnclose</code> .
N03	<code>validation/check_mathics_acceptance.py</code>	<code>canonical_sources</code> , <code>protocol_fields</code> , and <code>validate_shard</code> .
Novelty	<code>docs/development/CODE_REVIEW_STATUS.md</code>	C20, C22, P08, and main- tained evidence bound- aries.
Novelty	<code>external-reports/code-review/wave-2/code-review-14/article.tex</code>	Original N02 affine- rounding and N05 quantitative-bound find- ings.
Novelty	<code>external-reports/code-review/wave-4/code-review-36/findings.json</code>	Original runner/aggrega- tion entries, especially N06.

B Acceptance assertions in compact form

For N01, the central postcondition is

retained transported contract after a no-op $\implies$ retained existing discard companion.

The finite expression, remainder, conditions, and quantitative upper bound should be unchanged under the same no-op. This does not require retaining a signed first-omitted-term certificate after a nontrivial truncation.

For N02, the first acceptance assertion is exact interval equality between old and changed evaluators on the supported grammar. The second is a structural counter bounded by a constant multiple of the input tree nodes, without recursive whole-subtree key construction. A native wall-clock claim requires an additional measured public workload and is not implied by either assertion.

For N03, existing entry-name admission, new path membership, and no-observed-drift are conjoined with all existing shard checks. They are not alternatives to source map equality, protocol checks, or complete case coverage. They establish consistency under the existing recorder trust model, not authenticity of an arbitrary submitted record.

References

- [1] Vladimir Reshetnikov and contributors. *Asymptotic*, reviewed revision 651f2029d0b2cd4da9e4dfdf1f4275a124d23b99. <https://github.com/VladimirReshetnikov/Asymptotic/tree/651f2029d0b2cd4da9e4dfdf1f4275a124d23b99>.
- [2] Repository source, *AsymptoticAnalysis.wl*, canonical kernel. <https://github.com/VladimirReshetnikov/Asymptotic/blob/651f2029d0b2cd4da9e4dfdf1f4275a124d23b99/src/Kernel/AsymptoticAnalysis.wl>.

- [3] Repository source, *SeriesOperations.wl*. <https://github.com/VladimirReshetnikov/Asymptotic/blob/651f2029d0b2cd4da9e4dfdf1f4275a124d23b99/src/Kernel/SeriesOperations.wl>.
- [4] Repository source, *DirichletSpecialFunctions.wl*. <https://github.com/VladimirReshetnikov/Asymptotic/blob/651f2029d0b2cd4da9e4dfdf1f4275a124d23b99/src/Kernel/DirichletSpecialFunctions.wl>.
- [5] Repository source, *InverseCertificates.wl*. <https://github.com/VladimirReshetnikov/Asymptotic/blob/651f2029d0b2cd4da9e4dfdf1f4275a124d23b99/src/Kernel/InverseCertificates.wl>.
- [6] Repository source, *check_mathics_acceptance.py*. https://github.com/VladimirReshetnikov/Asymptotic/blob/651f2029d0b2cd4da9e4dfdf1f4275a124d23b99/validation/check_mathics_acceptance.py.
- [7] Repository history, *Verify and record complete focused Mathics acceptance*, commit `e10ef1b71f5b55080c62f52c3ce9d3d6e039fc99`. <https://github.com/VladimirReshetnikov/Asymptotic/commit/e10ef1b71f5b55080c62f52c3ce9d3d6e039fc99>.
- [8] Repository review corpus index, *Code review reports*. <https://github.com/VladimirReshetnikov/Asymptotic/blob/651f2029d0b2cd4da9e4dfdf1f4275a124d23b99/external-reports/code-review/README.md>.
- [9] Maintained *Code review implementation status*. https://github.com/VladimirReshetnikov/Asymptotic/blob/651f2029d0b2cd4da9e4dfdf1f4275a124d23b99/docs/development/CODE_REVIEW_STATUS.md.
- [10] Maintained *Wave 3 intake*. https://github.com/VladimirReshetnikov/Asymptotic/blob/651f2029d0b2cd4da9e4dfdf1f4275a124d23b99/docs/development/WAVE_3_INTAKE.md.
- [11] Maintained *Wave 4 intake*. https://github.com/VladimirReshetnikov/Asymptotic/blob/651f2029d0b2cd4da9e4dfdf1f4275a124d23b99/docs/development/WAVE_4_INTAKE.md.
- [12] Original external report 14, *Incremental Independent Audit of AsymptoticInverse 1.8.0*, especially N02 and N05. <https://github.com/VladimirReshetnikov/Asymptotic/blob/651f2029d0b2cd4da9e4dfdf1f4275a124d23b99/external-reports/code-review/wave-2/code-review-14/article.tex>.
- [13] Original external report 36, finding ledger. <https://github.com/VladimirReshetnikov/Asymptotic/blob/651f2029d0b2cd4da9e4dfdf1f4275a124d23b99/external-reports/code-review/wave-4/code-review-36/findings.json>.